

Framework Adaptation and Architectural Rationale

Project Overview

My *PocketLLM Portal* architecture has been adapted from a framework-independent specification (Assignment 2) into a concrete implementation using:

- **Frontend:** React.js
- **Backend:** Node.js + Fastify
- **Database:** SQLite (file-based persistence)

This adaptation ensures that all views (Logical, Component, Deployment, Package, and Sequence (Single and Stream)) conform to each chosen technology's structural and behavioral conventions while maintaining architectural clarity across tiers.

➤ Framework Selection Rationale

React (Presentation Tier)

- **Component-based structure:**
The logical UI elements from Assignment 2 were converted into <React Component> classes—App, PromptInput, ModeSelector, ChatView, and MetricsPanel—to align with React's declarative and compositional paradigm.
- **Reactive state handling:**
React's useState and useEffect hooks replace abstract "updateUI()" logic, ensuring automatic DOM updates and metric polling.
- **Frontend API abstraction:**
The former generic "HTTPClient" was re-designed as IFetchClient, encapsulating both REST and Server-Sent Event (SSE) requests using browser-native Fetch and EventSource APIs.

Fastify (Application Tier)

- **High-performance Node framework:**
Fastify was selected for its lightweight async model and declarative route registration, directly mapping to the controller elements (GenerateRoute, StreamRoute, MetricsRoute).
- **Plugin-based extensibility:**
The FastifyApp class represents the root server object that registers controllers, paralleling the "Application Controller" pattern in Assignment 2.
- **Built-in JSON serialization:**
Simplifies REST/SSE communication with React's FetchClient and maintains stateless request handling.

SQLite (Data Tier)

- **Self-contained persistence layer:**
SQLite replaces the abstract “PersistenceService” with a concrete SQLiteRepository, storing cached completions and metric logs in a single local database file (cache.db).
- **Ease of deployment:**
Eliminates the need for external database setup—ideal for portable demos or offline execution.
- **Direct SQL access:**
The CacheService delegates persistence operations via SQL read/write methods to SQLiteRepository.

➤ Major Adaptations from Assignment 2

Element (A2)	Framework-Specific Adaptation (A3)	Rationale
UIController, PromptHandler	→ React Components (PromptInput, App)	React enforces a hierarchical component tree; event handlers are bound to components rather than global controllers.
HTTPService	→ IFetchClient Interface	Browser environment replaces network abstraction with Fetch and EventSource APIs.
AppServer	→ FastifyApp	Fastify bootstraps through .register() routes, mirroring controller pattern.
Controller.generate()	→ GenerateRoute	Each API endpoint (POST /api/v1/generate) becomes its own Fastify route.
DomainLogic	→ CacheService, MetricsService, HashingService, ModelAdapter	Segregated per SRP (Single Responsibility Principle) and mapped to Node modules.
PersistenceManager	→ SQLiteRepository	Implements concrete SQL I/O using SQLite for lightweight persistence.
StreamHandler	→ StreamRoute with SSE	React’s streaming mode integrated via Server-Sent Events to push tokens.
MonitorService	→ MetricsRoute + MetricsPanel	Split into backend metrics collection and frontend visualization.

➤ Compliance with Framework Conventions

Frontend Compliance

- Single root <App> component as entry point (React standard).
- Functional components + hooks instead of procedural control flow.
- Props-based data passing ensures loose coupling among child components.
- API abstraction (IFetchClient) maintains separation of concerns and testability.

Backend Compliance

- Each route (GenerateRoute, StreamRoute, MetricsRoute) registered via fastify.register().
- Services injected as dependencies—Fastify remains stateless and easily testable.

- Metrics and cache layers implemented as independent modules following Fastify's plugin-style modularity.

Data Tier Compliance

- Follows repository pattern for persistence—SQLiteRepository encapsulates SQL commands.
- Data integrity maintained via synchronous file I/O within Node runtime.
- Supports horizontal replacement if migrated to MySQL/PostgreSQL later.

➤ Cross-View Consistency

- **Logical ⇔ Component View:** all classes and interfaces map one-to-one with components or controllers.
- **Component ⇔ Sequence View:** sequence diagrams validate runtime collaboration between PromptInput, GenerateRoute, and CacheService.
- **Deployment View:** physically allocates React (browser) and Fastify + SQLite (server) into separate nodes, connected via REST + SSE.
- **Package View:** folder organization mirrors runtime module dependencies—frontend/components, backend/routes, backend/services, infra/sqlite.

➤ Architectural Benefits

- **Clear Separation of Concerns:**
React handles user experience; Fastify manages API orchestration; SQLite persists data.
- **Low Latency Interaction:**
SSE (Server-Sent Events) ensures token streaming with near-real-time feedback.
- **Portability and Testability:**
The entire system runs locally with no external dependencies.
- **Maintainability:**
Consistent layering (Presentation → Application → Domain → Infrastructure) simplifies debugging and incremental upgrades.

➤ Summary

This framework-specific adaptation transforms the original technology-agnostic design into a directly implementable architecture compatible with **React, Fastify, and SQLite**.

Each diagram now faithfully reflects the concrete runtime structure, preserving architectural principles from Assignment 2 while embracing real-world software constraints such as component lifecycle, route registration, asynchronous I/O, and file-based persistence.

Changes Since Assignment 2

Instructor Feedback

- No comments were received for Assignment 2.
Therefore, the architectural evolution in Assignment 3 is based entirely on my own continuation, interpretation, and framework-specific refinements of the Assignment 2 design.

➤ Overview of Evolution

Assignment 2 established a **framework-independent, layered architecture** (Presentation, Application, Domain, Infrastructure).
Assignment 3 transitions that abstract model into a **concrete, implementable architecture** grounded in actual frameworks — **React (frontend)**, **Fastify (backend)**, and **SQLite (persistence)** — while preserving the original separation of concerns.

➤ Structural and Behavioral Changes

Aspect	Assignment 2 (Framework-Independent)	Assignment 3 (Framework-Specific)	Purpose of Change
Frontend Layer	Generic UIController and PromptHandler components with abstract event loops.	Concrete React components: App, PromptInput, ModeSelector, ChatView, and MetricsPanel.	Aligns with React’s component-based paradigm and state-driven rendering.
Network Communication	Abstract HTTPService for sending requests.	Introduced IFetchClient abstraction utilizing browser fetch() and EventSource.	Complies with browser-side API standards; supports both synchronous and streaming interactions.
Application Controller	Single generic AppServer managing routes.	FastifyApp with modular route handlers (GenerateRoute, StreamRoute, MetricsRoute).	Implements Fastify’s declarative route registration; enables modular scaling.
Prompt Processing	Monolithic DomainLogic handling text generation, caching, and metrics.	Decomposed into independent services: CacheService, MetricsService, HashingService, and ModelAdapter.	Applies the <i>Single Responsibility Principle</i> and improves testability.
Persistence	Abstract PersistenceManager using a generic data store.	SQLiteRepository with direct SQL I/O and on-disk persistence.	Chosen for lightweight deployment, offline operation, and minimal setup overhead.
Streaming Capability	Represented as a future extension in design notes.	Implemented explicitly as StreamRoute and openSSE() client logic.	Demonstrates real-time token streaming via Server-Sent Events (SSE).
Monitoring & Metrics	Conceptual “MonitorService.”	Split into backend MetricsService and frontend MetricsPanel.	Enables runtime observability and system health visualization.
Logical Modeling	UML diagrams were abstract (no tech references).	All diagrams are framework-bound: React UI, Fastify routes, SQLite persistence.	Ensures model consistency with the chosen technology stack.

➤ Diagram-Level Changes

Diagram Type	New/Modified Elements	Rationale
Logical Class Diagram	Added framework-specific classes (FastifyApp, IFetchClient, SQLiteRepository).	Represent real runtime classes replacing abstract placeholders.

Component Diagram	Split into Frontend (React) and Backend (Fastify) components.	Shows clearer deployment responsibility boundaries.
Package Diagram	Organized into four packages: frontend, backend.routes, backend.services, infra.sqlite.	Aligns with actual project folder hierarchy.
Sequence Diagrams (Single/Stream)	Added REST flow for /generate and SSE flow for /stream.	Captures true execution path differences between synchronous and streaming prompts.
Deployment Diagram	Includes two physical nodes: <i>Browser</i> (React) and <i>Server</i> (Fastify + SQLite).	Matches implementation setup and runtime separation.

- Key Architectural Improvements
- **Explicit technology mapping:** each logical entity now corresponds to an actual runtime construct.
 - **Improved modularity:** services like caching and metrics can be reused or replaced without altering business logic.
 - **True end-to-end traceability:** every frontend interaction now maps to backend processing and data persistence through well-defined routes.
 - **Enhanced performance modeling:** streaming and caching behaviors are now measurable through real metrics endpoints.
- Conclusion

Assignment 3 represents the **natural progression and materialization** of the architectural intent established in Assignment 2.

The transition from abstract layers to concrete framework modules demonstrates a practical, implementable architecture suitable for real-world deployment while adhering to the design principles introduced throughout the course.

Summary and Conclusion

The *PocketLLM Portal* architecture now represents a fully realized, framework-conformant system that bridges the conceptual design of Assignment 2 with a concrete, deployable implementation.

Through its layered structure—React frontend, Fastify backend, and SQLite persistence layer—the design achieves a balanced blend of **modularity**, **performance**, and **maintainability**.

The adaptation demonstrates how abstract UML models can evolve into tangible runtime entities without violating architectural principles such as *separation of concerns* and *single responsibility*.

Each diagram collectively reinforces this traceability:

- the **Logical Class Diagram** captures key domain abstractions,
- the **Component and Package Diagrams** illustrate the structural composition of React and Fastify modules,
- the **Sequence Diagrams** validate functional workflows for both single and streaming interactions,
- and the **Deployment Diagram** anchors the system in a realistic runtime topology.

Together, these artifacts showcase an architecture that is not only technically feasible but also extensible—ready for future scaling to distributed environments or API-based LLM integrations.

The final framework-specific model faithfully preserves the intent of the original design while grounding it in modern, production-ready technologies, thereby demonstrating the full architectural lifecycle from abstract specification to executable realization.